

MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: Minimum Spanning Tree - KruskalMST
Roll No. : MCA2511	Date : 08-12-2025

1. KruskalMST

Code:

```
import java.util.Scanner;
```

```
class Edge
```

```
{  
    int src;  
    int dest;  
    int weight;  
} //end of Edge class
```

```
public class KruskalMST
```

```
{  
    //Find operation for Union-Find  
    static int find(int parent[], int i)  
    {  
        while(parent[i] != i)  
        {  
            i = parent[i];  
        }  
        return i;  
    } //end of find method  
  
    //Union operation for Union-Find  
    static void union(int parent[], int x, int y)  
    {  
        int x_set = find(parent,x);  
        int y_set = find(parent,y);  
        parent[y_set] = x_set;  
    } //end of union method
```

```
    public static void main(String[] args)
```

```
    {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("Enter number of vertices: ");  
        int n = sc.nextInt();
```

```
int[][] adj = new int[n][n];

System.out.println("Enter the Adjacency Matrix - 0 for no edge");
for(int i=0; i<n;i++)
{
    for(int j=0; j<n; j++)
    {
        adj[i][j] = sc.nextInt();
    }
}

//create Edge List from Adjacency Matrix
Edge[] edges = new Edge[n*n];
int edgeCount = 0;

for(int i = 0; i<n; i++)
{
    for(int j=i+1; j<n; j++)
    {
        if(adj[i][j] != 0)
        {
            Edge e = new Edge();
            e.src = i;
            e.dest = j;
            e.weight = adj[i][j];
            edges[edgeCount] = e;
            edgeCount++;
        }
    }
}

//Sort
for(int i = 0; i<edgeCount-1; i++)
{
    for(int j = 0; j<edgeCount-i-1; j++)
    {
        if(edges[j].weight > edges[j+1].weight)
        {
            Edge tmp = edges[j];
            edges[j] = edges[j+1];
            edges[j+1] = tmp;
        }
    }
}

//Kruskal's Algorithm
```

```
int[] parent = new int[n];
for(int i = 0; i<n; i++)
{
    parent[i] = i;          //Each vertex is its own parent initially
}

Edge[] mst = new Edge[n-1];
int mstEdges = 0;
int i=0;
int total=0;

while(mstEdges<n-1 && i<edgeCount)
{
    Edge nextEdge = edges[i++];
    int x = find(parent,nextEdge.src);
    int y = find(parent, nextEdge.dest);

    //If including this edge does not cause a cycle
    if(x!=y)
    {
        mst[mstEdges] = nextEdge;
        mstEdges++;
        total += nextEdge.weight;
        union(parent,x,y);
    }
}

//end of while

//Print the MST
System.out.println("\nEdges int the MST: ");
for(i = 0; i<mstEdges; i++)
{
    System.out.println("Vertex "+mst[i].src + " - Vertex " + mst[i].dest + "
Weight = " + mst[i].weight);
}
System.out.println("Total weight of MST: " + total);
}

//end of psvm
}

//end of class KruskalMST
```

Output:

```
A:\MCA2511\DS_LAB>javac KruskalMST.java
```

```
A:\MCA2511\DS_LAB>java KruskalMST
```

```
Enter number of vertices:
```

```
5
```

```
Enter the Adjacency Matrix - 0 for no edge
```

```
0 20 50 0 0
```

```
20 0 20 0 30
```

```
50 20 0 40 0
```

```
0 0 40 0 20
```

```
0 30 0 20 0
```

```
Edges int the MST:
```

```
Vertex 0 - Vertex 1 Weight = 20
```

```
Vertex 1 - Vertex 2 Weight = 20
```

```
Vertex 3 - Vertex 4 Weight = 20
```

```
Vertex 1 - Vertex 4 Weight = 30
```

```
Total weight of MST: 90
```